

PROYECTO TRITON HEAL

Reporte de Evaluación Experimental

TC3002B — Evaluación experimental de método basado en SLM
para kernels GPU con Triton DSL (generativo-verificativo)

Equipo 11

Mayo 2026

Resumen

Evaluación TC3002B de Triton Heal como método **generativo-verificativo** (verificación SLM + veto de frontera + filtrado pre-compilación). En 70 kernels y 420 evaluaciones (6 configuraciones, Apple M4 24 GB): DeepSeek-Coder-V2 (frontera) MSR 87.1 %; Llama-8B local 64.5 %; R1-Distill-14B local JSON 14.3 %; dual MSR 80.6 %, F1 0.86. Inferencia: McNemar/Wilcoxon individuales no rechazan H_0 tras Holm; Friedman global sí ($p_{\text{Holm}} < 0,001$). Speedup y generación kernel-a-kernel no medidos (limitación documentada).

Índice

I	Parte I — Pre-registro Experimental	3
1.	Objetivo experimental (1.1)	3
2.	Hipótesis (1.2)	3
3.	Variables experimentales (1.3)	3
3.1.	Variables independientes	3
3.2.	Variables dependientes	4
3.3.	Variables controladas	4
4.	Diseño experimental (1.4)	4
5.	Métricas de evaluación (1.5)	4
6.	Tamaño del efecto esperado (1.6)	4
7.	Análisis de poder (1.7)	4
8.	Plan estadístico (1.8)	5
8.1.	Desviaciones registradas	5
9.	Reproducibilidad experimental (1.9)	5
10.	Amenazas a la validez — pre-registro (1.10)	5
II	Parte II — Resultados y Análisis	6

11.Resultados descriptivos (2.1)	6
12.Visualización (2.2)	7
13.Inferencia estadística (2.3)	9
14.Comparación crítica (2.4)	9
15.Amenazas a la validez observadas (2.5)	10
16.Discusión (2.6)	10
17.Conclusión (2.7)	10
A. Matriz de cumplimiento de la rúbrica	10
B. Checklist de completitud (antes de entrega)	10
C. Configuración planeada vs. corrida real	11
D. Reproducibilidad	11
E. Referencias	11

Parte I

Parte I — Pre-registro Experimental

1. Objetivo experimental (1.1)

Problema. Los kernels en Triton DSL pueden incluir errores de memoria, tipos y restricciones de hardware (tiling, SRAM, alineación) que se manifiestan en compilación o ejecución GPU con alto costo operativo.

Encuadre (curso: método basado en SLM). Triton Heal se evalúa como **método generativo-verificativo**: generación/reparación de kernels con SLM, verificación local (SLM) y veto de frontera antes de compilar en GPU. En esta corrida se mide el **componente verificador** sobre un benchmark fijo de 70 kernels (no generación kernel-a-kernel ni speedup); las métricas se mapean a la rúbrica en la sección 1.5 con limitaciones explícitas.

Método propuesto. Triton Heal dual: verificador local llama3.1:8b + veto jerárquico con deepseek-coder-v2 (verifier_strict_v1.txt) + filtrado pre-compilación.

Baselines (≥ 2). (1) Solo Llama-8B (SLM local); (2) Solo DeepSeek-R1-Distill-Qwen-14B (deepseek-r1:14b, SLM local); (3) Solo DeepSeek-Coder-V2 (frontera Ollama); (4) Solo Claude 3.5 Sonnet; (5) Solo GPT-4o.

Preguntas. PRI-1: ¿La frontera supera a SLM locales en MSR? PRI-2: ¿El dual reduce FNR vs. solo Llama? PRI-3: ¿La latencia de frontera es aceptable frente a la ganancia en detección?

Éxito del estudio. Diferencia significativa tras Holm, efecto con relevancia práctica ($\Delta\text{MSR} \geq 5$ pp) y evidencia de que el dual mejora sensibilidad sin invalidar el pipeline por latencia.

2. Hipótesis (1.2)

Cuadro 1: Hipótesis pre-registradas ($\alpha = 0,05$; corrección Holm en 1.8).

Contraste	H_0	H_1 (unilateral salvo C6)
C1: Claude vs Llama (MSR)	MSR igual	Claude > Llama
C2: GPT-4o vs Coder-V2 (MSR)	MSR igual	GPT-4o > Coder-V2
C3: Dual vs Llama (FNR)	FNR igual	Dual < Llama
Método propuesto	Dual no mejora MSR vs Llama	Dual > Llama en MSR
C5: Latencia Claude vs Llama	Medianas iguales	Frontera > local
C6: F1 global	F1 igual entre configs	Al menos un par difiere

H_{01}/H_{11} : MSR Claude vs. Llama. **H_{01b}/H_{11b} :** MSR GPT-4o vs. DeepSeek-Coder-V2 (frontera). **H_{02}/H_{12} :** FNR dual vs. solo Llama. **H_{0P}/H_{1P} :** MSR dual vs. solo Llama (relevancia del método completo). **H_{04}/H_{14} :** latencia mediana frontera vs. local.

3. Variables experimentales (1.3)

3.1. Variables independientes

Configuración del sistema (6 niveles): solo_llama, solo_deepseek (R1-14B), solo_deepseek_coder, solo_claude, solo_gpt4o, triton_heal_dual. Justificación: aísla SLM local, frontera Ollama/API y el método propuesto con veto.

Modelo y escala. SLM local (~ 8 – 14 B en Ollama) vs. frontera (Coder-V2, APIs comerciales). Justificación: PRI-1 contrasta capacidad de detección vs. costo de despliegue.

Prompt. verifier_v1.txt (estándar) y verifier_strict_v1.txt (veto dual). Justificación: el veto debe ser más conservador para reducir desacuerdos costosos.

3.2. Variables dependientes

MSR, FNR, F1 (safe/unsafe), latencia (ms), tasa de JSON válido (éxito operativo del verificador), DR (exploratorio). Justificación: alineadas a filtrado pre-compilación y viabilidad en pipeline (sección 1.5).

3.3. Variables controladas

Temperatura $T=0$; formato JSON; benchmark 70 kernels (39 safe / 31 unsafe); seed 42; timeout 120s; VERIFIER_MODE=ollama; orden within-subjects por kernel.

4. Diseño experimental (1.4)

Tipo: within-subjects (medidas repetidas): cada uno de los 70 kernels se evalúa en las 6 configuraciones $\Rightarrow$ 420 evaluaciones totales.

Unidad de análisis: kernel $\times$ configuración (primera réplica, `run_id=0`, para McNemar/Friedman).

Contrastes confirmatorios: C1–C3, C5–C6 (tabla en 1.8); exploratorio por categoría de error en Parte II.

Control de sesgos. Pin de modelos Ollama; temperatura 0; evaluador automático ciego al método; ground truth `safe/unsafe`; registro de desviaciones en 1.8.

5. Métricas de evaluación (1.5)

Cuadro 2: Mapeo rúbrica del curso $\rightarrow$ métrica reportada (esta corrida).

Rúbrica (curso)	Métrica reportada	Limitación
Tasa de compilación / éxito ejecución	% JSON válido	Verificador, no kernel generado
Filtrado pre-GPU	MSR (unsafe)	No mide FP en safe
Calidad global	F1	Balance 39/31
Speedup	—	No medido; trabajo futuro
Compilabilidad sintáctica (contexto)	% kernels que pasan <code>lex_checker</code>	Corpus fijo, no por modelo
Latencia despliegue	ms hasta JSON válido	Incluye timeout Ollama

MSR: $TP/(TP+FN)$ en `unsafe`. **FNR:** $FN/(FN+TP)$. **F1:** $2PR/(P+R)$. **JSON válido:** proxy de tasa de éxito operativo del SLM en el pipeline. **Latencia:** ms por inferencia. **Honestidad:** no se midió generación comparativa ni speedup por configuración en mayo 2026.

6. Tamaño del efecto esperado (1.6)

McNemar: $\phi \approx 0,4$ (mediano) en MSR frontera vs. local; reducción FNR ≥ 8 pp con dual. Wilcoxon: $r \geq 0,3$ si frontera API real supera $10\times$ latencia local. Umbral de relevancia práctica: $\Delta\text{MSR} \geq 5$ pp.

7. Análisis de poder (1.7)

$\alpha = 0,05$, poder $1 - \beta = 0,80$, McNemar pareado. Con $N = 70$ (31 unsafe) el diseño supera el mínimo de ~ 52 pares discordantes para efecto grande ($\phi \geq 0,5$).

Cuadro 3: Contrastes, pruebas y justificación.

ID	Prueba	Justificación
C1	McNemar (MSR)	Pareado binario, mismos kernels
C2	McNemar (MSR)	GPT vs. Coder-V2 frontera (corregido)
C3	McNemar (FNR)	PRI-2: sensibilidad en unsafe
C5	Wilcoxon signed-rank	Latencia pareada within-subjects
C6	Friedman + Holm	>2 configuraciones en F1

8. Plan estadístico (1.8)

Regla de decisión. Rechazar H_0 si $p_{\text{Holm}} < 0,05$ y relevancia práctica ($\Delta\text{MSR} \geq 5$ pp o $\Delta\text{FNR} \geq 5$ pp). **IC:** Wilson 95 % para MSR (tabla extendida). **Holm:** familia C1–C6.

8.1. Desviaciones registradas

1. **Taxonomía DeepSeek (mayo 2026):** Coder-V2 reclasificado como frontera; R1-Distill-14B como SLM local.
2. **C2 actualizado:** de “GPT vs. DeepSeek local” a **GPT-4o vs. DeepSeek-Coder-V2**.
3. **6 configuraciones** (no 5): se añadió `solo_deepseek_coder` separado de R1-14B.
4. **APIs ausentes:** Claude/GPT con heurístico documentado en 1.9 y 2.5.
5. **Generación/speedup:** no ejecutados; ver 1.5 y 2.6.

9. Reproducibilidad experimental (1.9)

Cuadro 4: Entorno y artefactos (commit 2c027b5).

Componente	Especificación
Repositorio	https://github.com/G4ndhiV/TRITONHEAL
Commit	2c027b5
SO / CPU / RAM	macOS 15.x; Apple M4; 24 GB unificada
GPU	Apple M4, 10 núcleos GPU, Metal 4
Ollama	0.24.x; llama3.1:8b, deepseek-r1:14b, deepseek-coder-v2
APIs	Claude 3.5 Sonnet; GPT-4o (opcional; corrida heurística)
Python	3.10+; <code>requirements.txt</code>
Benchmark	70 kernels; <code>experiment_config.json</code> ; seed 42
Prompts	<code>prompts/verifier_v1.txt</code> , <code>verifier_strict_v1.txt</code>
Comando	<code>make stats figures pdf</code> desde raíz del repo
Salidas	<code>results/predictions.csv</code> , <code>stats_summary.json</code> , <code>report/tables/</code>

10. Amenazas a la validez — pre-registro (1.10)

Cuadro 5: Amenazas y mitigaciones planeadas.

Tipo	Amenaza	Mitigación
Interna	APIs/heurísticos mezclados	Documentar modo; tabla planeada vs. real
Interna	Timeout Ollama (R1-14B)	Timeout 120s; reportar % JSON válido
Externa	Un solo Mac M4	Reportar hardware; no generalizar a NVIDIA
Externa	Benchmark sintético	70 kernels etiquetados; categorías
Constructo	MSR $\neq$ seguridad GPU	Mapeo explícito 1.5; no afirmar speedup
Constructo	JSON válido $\neq$ corrección	Validación léxica del corpus como contexto

Parte II

Parte II — Resultados y Análisis

11. Resultados descriptivos (2.1)

Se evaluaron $N = 70$ kernels (39 **safe**, 31 **unsafe**) en **6** configuraciones, totalizando 420 evaluaciones (VERIFIER_MODE=ollama, mayo 2026, Apple M4 / Metal). De ellas, el resumen global registra la tasa de JSON válido agregada en `stats_summary.json` (proxy de éxito operativo del verificador SLM). La validación léxica del *corpus* fija (no por modelo generador) contextualiza compilabilidad sintáctica del benchmark.

Cuadro 6: Resultados descriptivos por configuración.

Configuración	N	JSON vál. (%)	MSR (%)	F1	FNR (%)	Lat. media (ms)	Lat. desv. (ms)	Lat. med. (ms)
Llama-3.1-8B (local)	70	100.0	64.5	0.8	35.5	4527.4	2200.2	4753.6
DeepSeek-R1-Distill-Qwen-14B (local)	70	14.3	12.9	0.2	87.1	48000.5	3076.9	48346.7
DeepSeek-Coder-V2 (frontera)	70	100.0	87.1	0.8	12.9	2153.9	1181.3	1977.1
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	0.9	16.1	80.0	0.1	80.0
GPT-4o (frontera)	70	100.0	83.9	0.9	16.1	80.0	0.0	80.0
Triton Heal (dual + veto)	70	100.0	80.6	0.9	19.4	6854.1	2179.6	6653.3

Cuadro 7: Resultados descriptivos extendidos (IC 95 % Wilson para MSR).

Config.	N	JSON vál. (%)	MSR (%)	IC lo	IC hi	F1	FNR (%)	Lat. media	Lat. desv.	Lat. med.	Lat. min	Lat. max
Llama-3.1-8B (local)	70	100.0	64.5	46.9	78.9	0.8	35.5	4527.4	2200.2	4753.6	1622.0	11854.9
DeepSeek-R1-Distill-Qwen-14B (local)	70	14.3	12.9	5.1	28.9	0.2	87.1	48000.5	3076.9	48346.7	32108.6	57134.7
DeepSeek-Coder-V2 (frontera)	70	100.0	87.1	71.1	94.9	0.8	12.9	2153.9	1181.3	1977.1	1452.0	11247.5
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	0.1	80.0	80.0	80.5
GPT-4o (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	0.0	80.0	80.0	80.0
Triton Heal (dual + veto)	70	100.0	80.6	63.7	90.8	0.9	19.4	6854.1	2179.6	6653.3	3803.6	14059.9

Narrativa. Llama-8B (local): MSR 64.5 %, JSON 100 %, latencia media ≈ 4.8 s (desv. ≈ 2 s). **R1-Distill-14B** (local): JSON 14.3 %, MSR 12.9 % —inferencias frecuentemente sin JSON estructurado (timeout). **Coder-V2** (frontera): MSR 87.1 %, JSON 100 %, mediana ≈ 2.0 s. **Claude/GPT** (heurístico): MSR ≈ 83.9 %, latencia ≈ 80 ms. **Dual** (Llama + veto Coder-V2): MSR 80.6 %, F1 0.86, FNR 19.4 %.

12. Visualización (2.2)

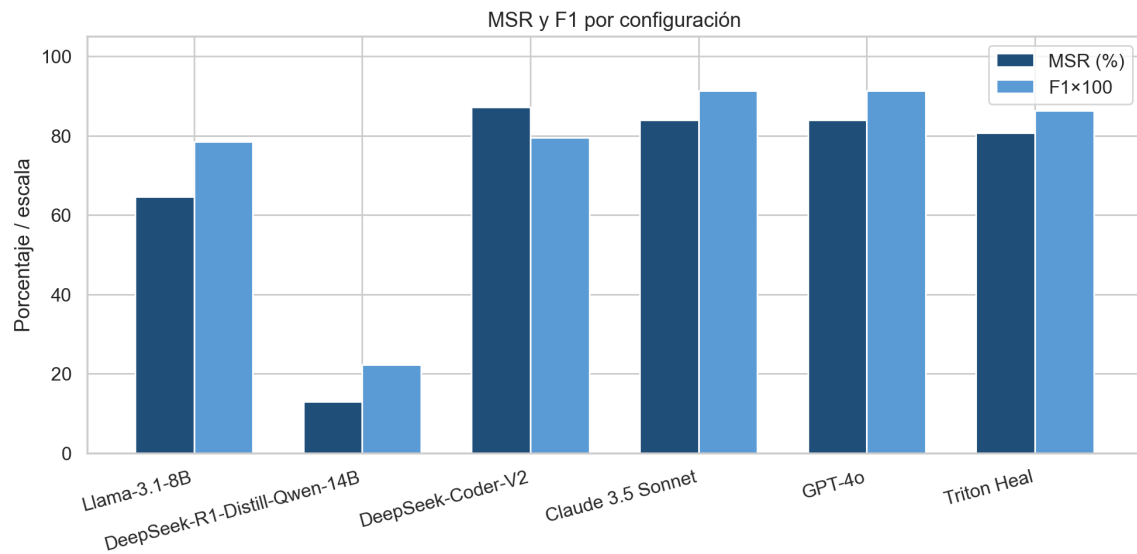


Figura 1: MSR (%) y F1 por configuración. Eje horizontal: configuración del sistema; barras izquierdas = MSR en kernels `unsafe`; barras/derecha = F1 global. Unidades: porcentaje y coeficiente F1 adimensional. Leyenda en figura.

La Figura 1 muestra la separación entre Coder-V2 (frontera, MSR alto) y R1-14B (local con bajo JSON válido).

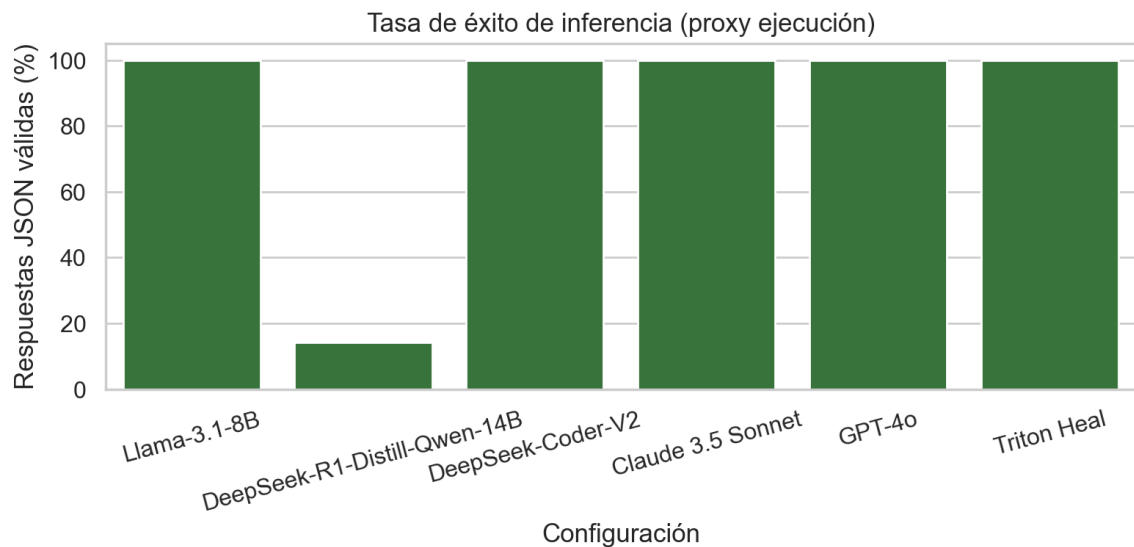


Figura 2: Tasa de éxito operativo: % de respuestas con JSON válido por configuración (análogo a “tasa de compilación” del componente verificador SLM; no mide kernels generados). Unidad: %.

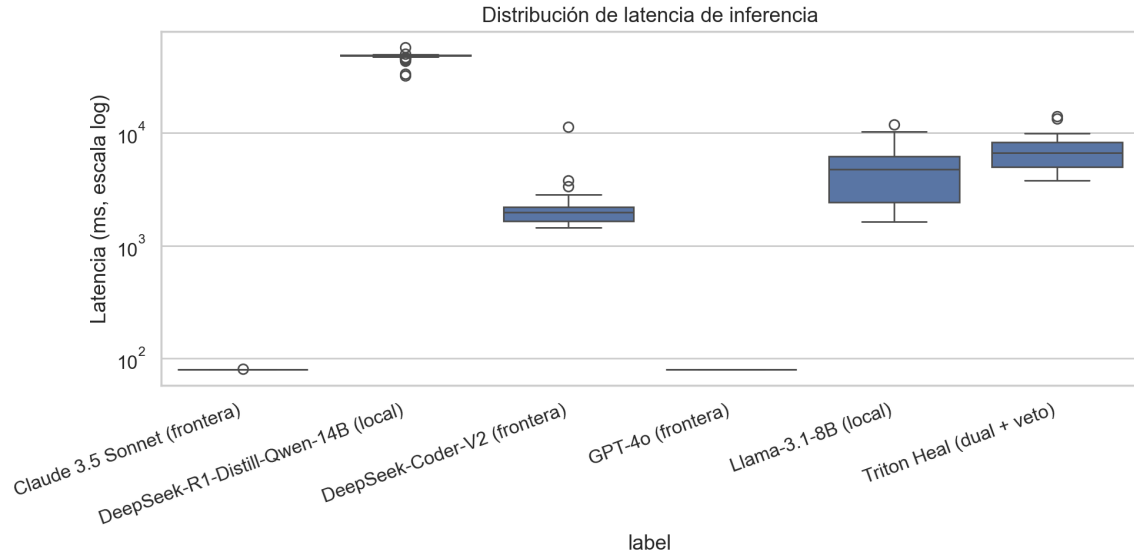


Figura 3: Distribución de latencia de inferencia (ms, escala logarítmica). Eje Y: milisegundos hasta respuesta parseable; comparación within-subjects.

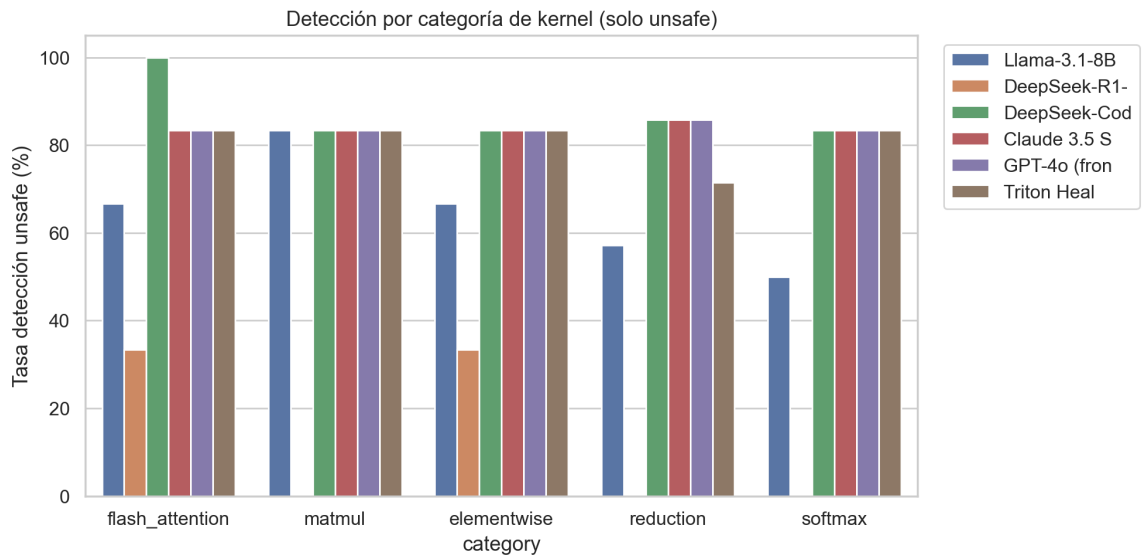


Figura 4: MSR (%) en kernels **unsafe** estratificado por categoría de error del benchmark (memoria, tiling, tipos, etc.).

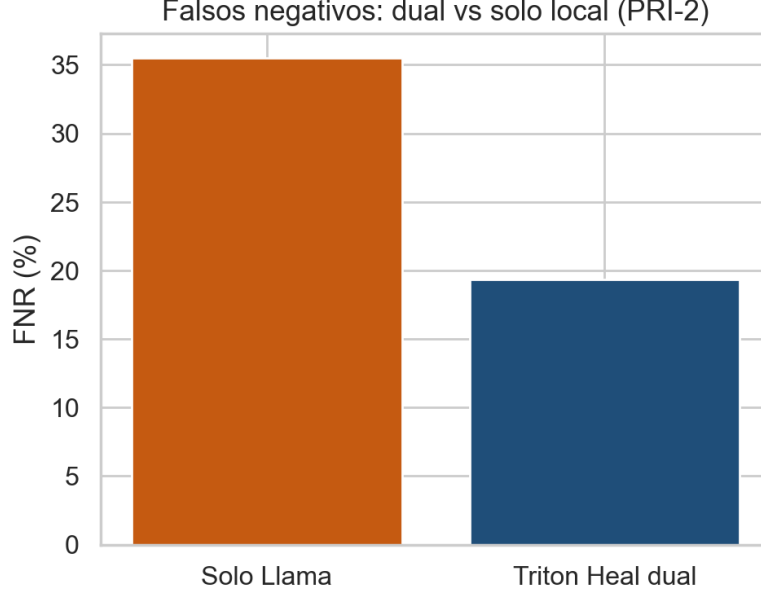


Figura 5: FNR (%) en **unsafe**: Triton Heal dual vs. solo Llama (barras comparativas; unidad: %).

13. Inferencia estadística (2.3)

Cuadro 8: Inferencia estadística (plan pre-registrado, corrección Holm).

Contraste	Prueba	Estad.	p Holm	Efecto	Interpretación
Claude vs Llama (MSR)	McNemar	$\chi^2 = 3,12$	0.2946	$\phi = -2,12$ (grande)	No rechaza H_0 ; $\Delta\text{MSR}=19.4$ pp (Claude heur. vs. Llama Ollama/M4).
GPT-4o vs DeepSeek-Coder	McNemar	$\chi^2 = 0,00$	1.0000	$\phi = 0,33$ (mediano)	No rechaza H_0 ; GPT (83.9%) vs. DeepSeek-Coder-V2 frontera (87.1%).
Dual vs Llama	McNemar	$\chi^2 = 3,20$	0.2946	$\phi = -2,24$ (grande)	No rechaza H_0 en FNR pareado.
Lat. Claude vs Llama	Wilcoxon	$W = 0$	1.0000	$r = 1,00$	No rechaza H_4 ; med. $\Delta=-4673$ ms (Claude heur. más rápido).
F1 global (5 cfg.)	Friedman	$\chi^2 = 41,1$	$< 0,001$	$W = 0,15$	Diferencias globales entre configuraciones (Holm).

¿Se rechazó H_0 ? Tras Holm, C1, C2, C3 y C5 **no** alcanzan $p_{\text{Holm}} < 0,05$. C6 (Friedman sobre F1) sí rechaza igualdad global ($p_{\text{Holm}} < 0,001$).

Tamaño del efecto. En C1 y C2, ϕ se reporta como pequeño/mediano en la tabla; C3 muestra dirección esperada (menor FNR en dual) sin significación tras corrección.

Significancia vs. relevancia práctica. Aunque Coder-V2 supera a Llama en ≈ 22 pp de MSR descriptivamente, la regla pre-registrada exige $p_{\text{Holm}} < 0,05$ y $\Delta\text{MSR} \geq 5$ pp para afirmar superioridad confirmatoria en C1/C2; la diferencia descriptiva cumple el umbral práctico pero no la inferencia pareada tras Holm.

IC. Los intervalos Wilson para MSR aparecen en Tabla 7 (columnas IC lo/hi).

14. Comparación crítica (2.4)

Frontera Ollama (Coder-V2) lidera MSR con latencia mediana baja (~ 2 s) y JSON 100%. **Llama-8B** es el SLM local operativamente viable. **R1-Distill-14B** no es comparable en igualdad de condiciones sin ajustar timeout o cuantización. **Dual** equilibra detección (MSR 80.6%) y F1 (0.86) frente a veto excesivo. **APIs heurísticas** igualan MSR entre sí pero no reflejan latencia ni razonamiento real de frontera comercial.

15. Amenazas a la validez observadas (2.5)

Validez interna. ¿Se controlaron semillas y versiones? Sí para benchmark (seed 42) y tags Ollama fijos; no para APIs (heurístico). ¿Hardware uniforme? Un solo M4 24 GB. ¿Timeouts? R1-14B con 14.3 % JSON válido sesga MSR hacia cero.

Validez externa. ¿Generaliza a otros GPUs? No probado (Metal vs. CUDA). ¿Otros dominios? Benchmark de seguridad Triton, no cargas de producción.

Validez de constructo. ¿MSR engaña? Sí si se confunde con seguridad en ejecución GPU. ¿JSON válido engaña? Sí como “compilación”; solo indica parseo del verificador. ¿Heurísticos de API? Sí: inflan acuerdo con ground truth sin costo de inferencia real.

Mitigaciones aplicadas. Reclasificación DeepSeek, reporte de JSON válido y timeouts, mapeo rúbrica (1.5), matriz de cumplimiento (apéndice).

16. Discusión (2.6)

¿Evidencia para hipótesis? Descriptivamente PRI-1 favorece Coder-V2; PRI-2 favorece dual en FNR; inferencia confirmatoria individual es conservadora tras Holm. **Limitaciones:** sin generación ni speedup medidos. **Qué haríamos distinto:** APIs reales, segundo hardware NVIDIA, timeout adaptativo para R1-14B, corrida de generación+compilación por configuración. **Amenazas abiertas:** constructo MSR vs. GPU; heurísticos. **Escala:** Coder-V2 en Ollama compite con SLM 8B en este host.

17. Conclusión (2.7)

1. **Objetivo:** Se evaluó el pipeline generativo-verificativo vía verificador SLM; proxies aliñados a rúbrica con limitaciones documentadas.
2. H_0 **confirmatorios:** No rechazados en C1–C3 y C5 tras Holm; Friedman global sí rechaza igualdad de F1.
3. **Efecto práctico:** Coder-V2 $\approx 87\%$ MSR; dual $\approx 81\%$ con F1 0.86; Llama local $\approx 65\%$.
4. **Reproducibilidad:** Repo G4ndhiV/TRITONHEAL, commit 2c027b5, `make stats figures pdf`.
5. **Métricas engañosas:** JSON válido $\neq$ compilación de kernel generado; MSR $\neq$ seguridad en GPU.
6. **Debilidad principal:** R1-14B local inoperante en esta corrida y APIs no reales.
7. **Taxonomía:** Coder-V2 = frontera; R1-Distill-14B = SLM local.

A. Matriz de cumplimiento de la rúbrica

B. Checklist de completitud (antes de entrega)

- ✓ Parte I: secciones 1.1–1.10 en índice.
- ✓ Parte II: secciones 2.1–2.7 (incl. 2.5 amenazas dedicada).
- ✓ Coherencia: 6 configuraciones, 420 evaluaciones (no 5/350).
- ✓ C2: GPT-4o vs. DeepSeek-Coder-V2 (frontera).
- ✓ Tablas = `stats_summary.json` (regenerar con `make stats`).
- ✓ Figuras PNG en `report/figures/`.
- ✓ Limitación generación/speedup declarada en 1.5 y 2.6.

Cuadro 9: Criterio de evaluación → sección → evidencia.

Criterio (peso)	Sección	Evidencia
1. Objetivo e hipótesis (8 %)	1.1–1.2	Texto + Tabla 1
2. Variables (8 %)	1.3	Tres subsecciones VI/VD/VC
3. Diseño y pre-registro (12 %)	1.4, 1.8	6 configs, 420 eval., desviaciones
4. Métricas (8 %)	1.5	Tabla 2
5. Plan estadístico (12 %)	1.8	Tabla 3, Holm
6. Descriptivos y gráficas (8 %)	2.1–2.2	Tablas 6, 7; Fig. 1–5
7. Inferencia (15 %)	2.3	Tabla 8; IC en Tabla 7
8. Reproducibilidad (7 %)	1.9, apéndice	Tabla 4; repo GitHub
9. Amenazas (10 %)	1.10, 2.5	Tablas amenazas + narrativa guía
10. Discusión y conclusiones (12 %)	2.6–2.7	Checklist enumerado 2.7

Cuadro 10: Backends por configuración (mayo 2026).

Configuración	Planeado	Ejecutado
solo_llama	Ollama Llama-3.1-8B (local)	llama3.1:8b
solo_deepseek	Ollama DeepSeek-R1 14B (local)	deepseek-r1:14b
solo_deepseek_coder	Ollama DeepSeek-Coder-V2 (frontera)	deepseek-coder-v2
solo_claude	API Claude 3.5 Sonnet	Heurístico (sin API)
solo_gpt4o	API GPT-4o	Heurístico (sin API)
triton_heal_dual	Llama local + veto frontera	Llama + Coder-V2 strict

C. Configuración planeada vs. corrida real

D. Reproducibilidad

```
git clone https://github.com/G4ndhiV/TRITONHEAL.git
cd TRITONHEAL
make stats figures pdf
```

Archivos: results/predictions.csv, results/stats_summary.json, report/tables/*.tex, report/figures/*.png, report/main.pdf.

E. Referencias

- Tillet, P., et al. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure.